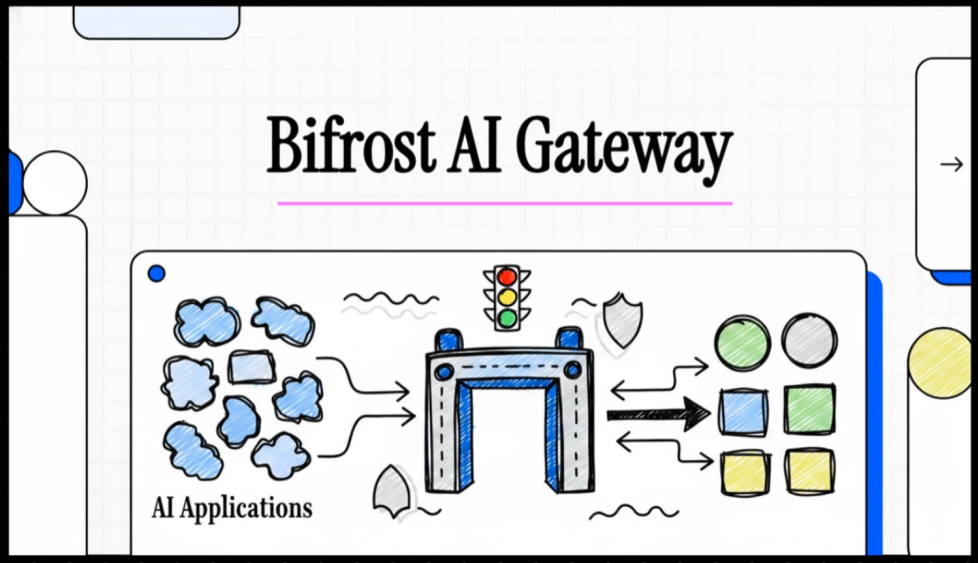
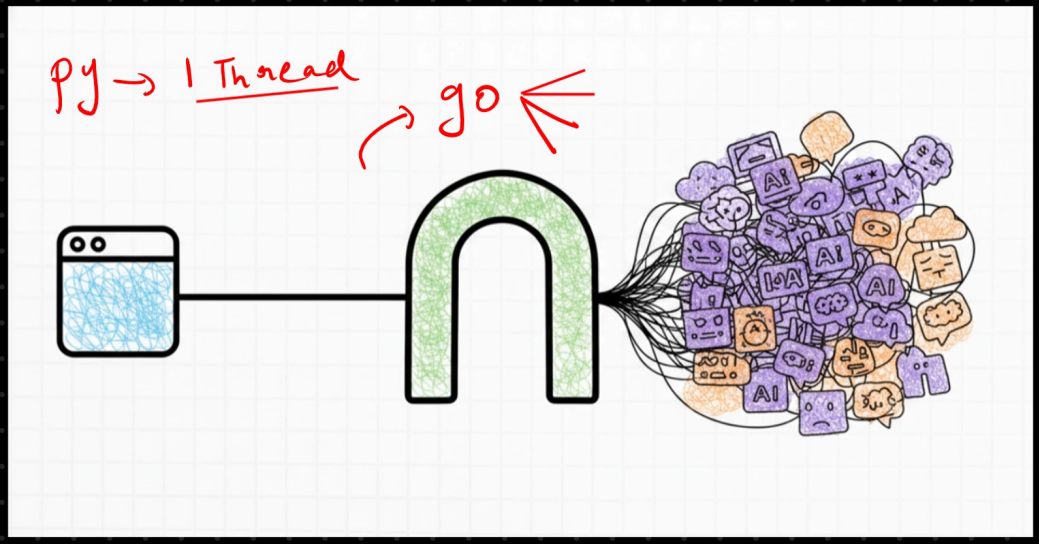
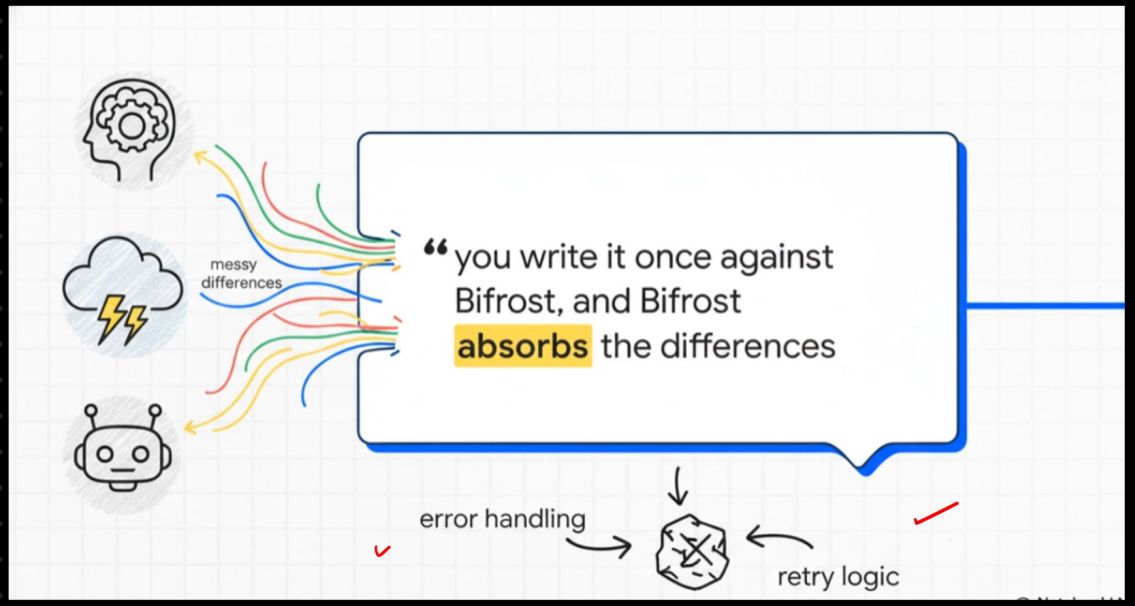
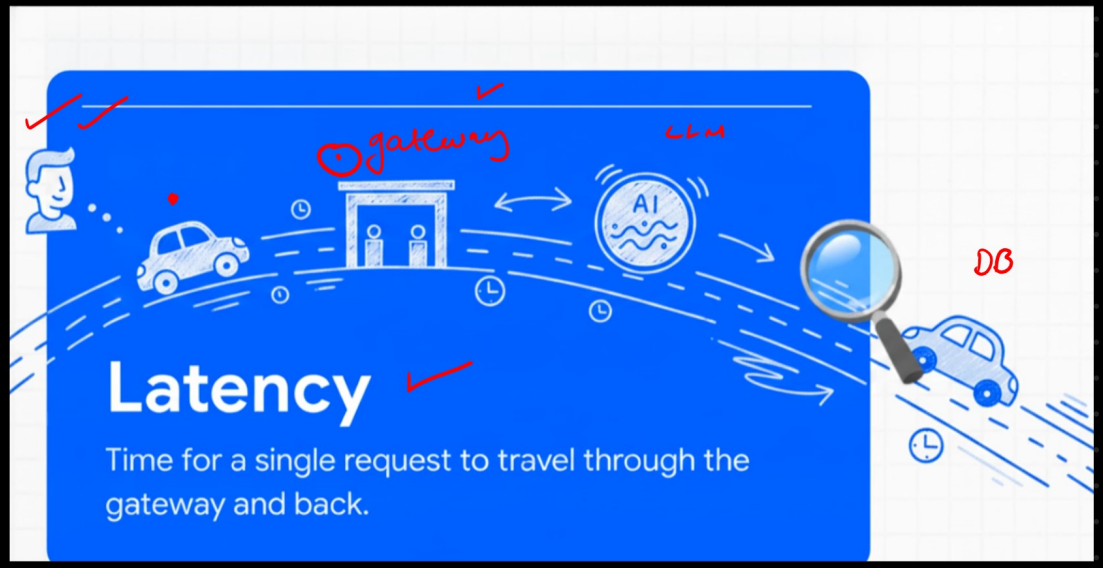
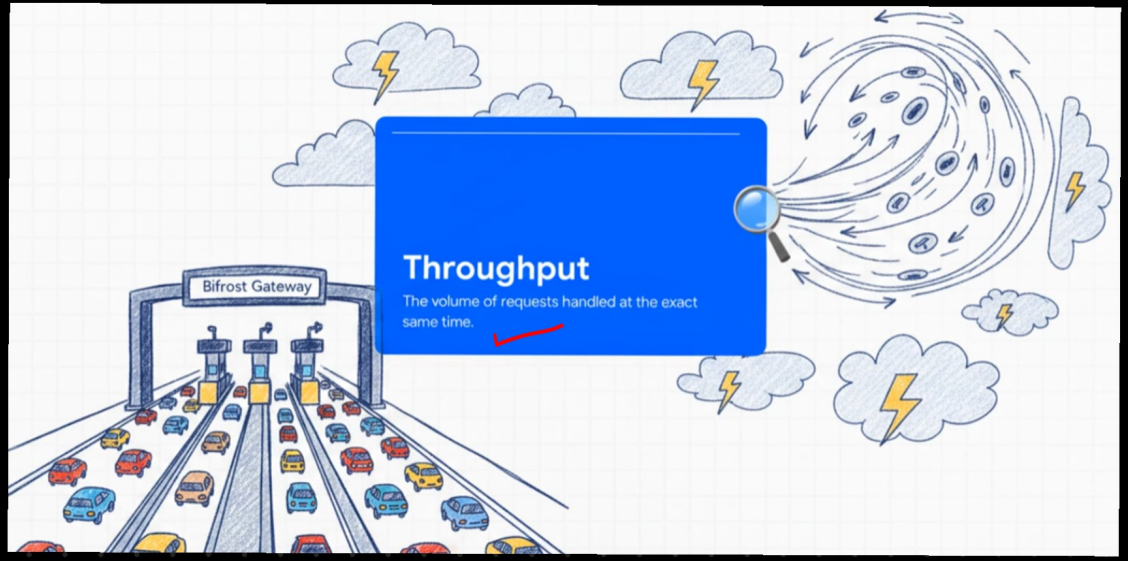

Bifrost AI Gateway
AI Applications


API
API
How to connect to
1,000 models safely?
AI

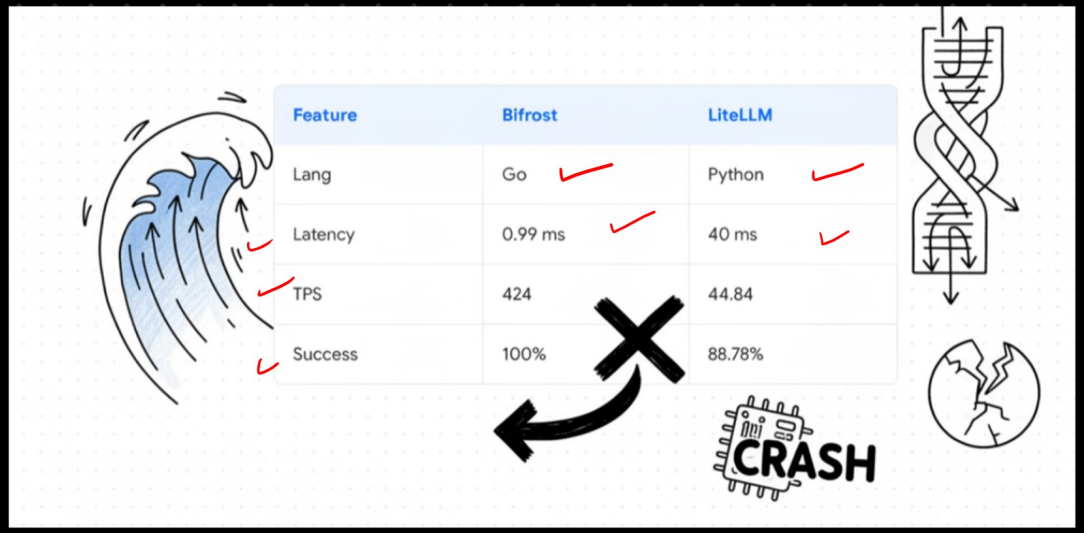

Py → 1 Thread
go

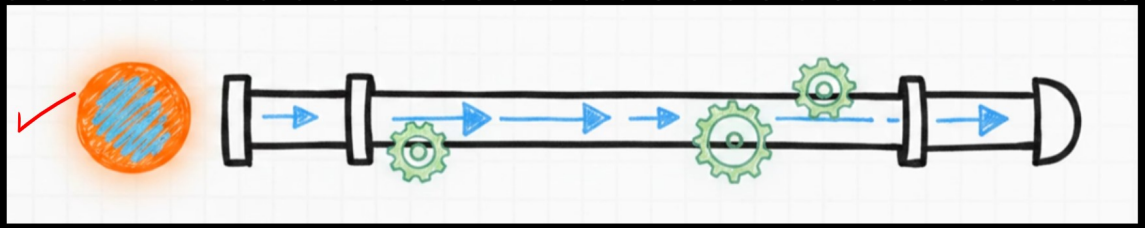

messy differences
“you write it once against Bifrost, and Bifrost absorbs the differences
error handling
retry logic

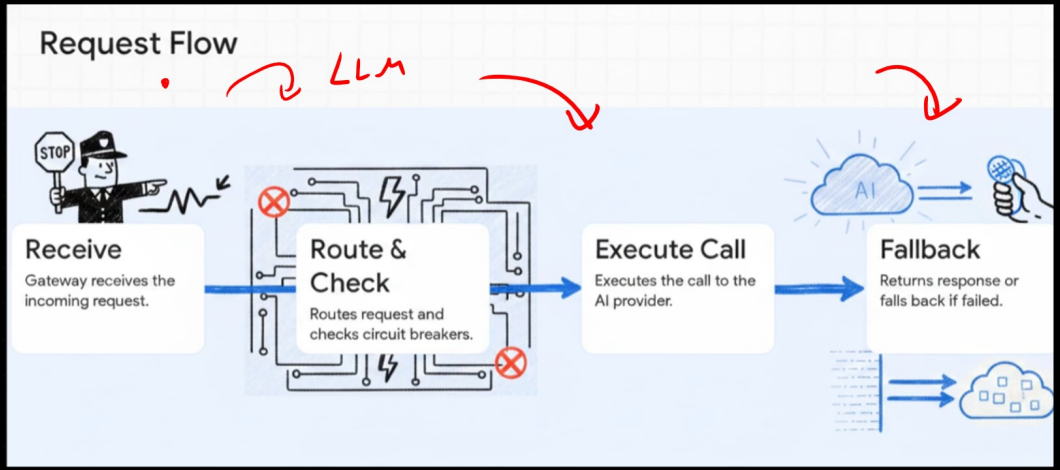

gateway
LLM
AI
DB
Latency
Time for a single request to travel through the gateway and back.


Bifrost Gateway
Throughput
The volume of requests handled at the exact same time.


Success Rate
The percentage of requests that succeed without failing.


Feature
Bifrost
LiteLLM
Lang
Go
Python
Latency
0.99 ms
40 ms
TPS
424
44.84
Success
100%
88.78%
CRASH

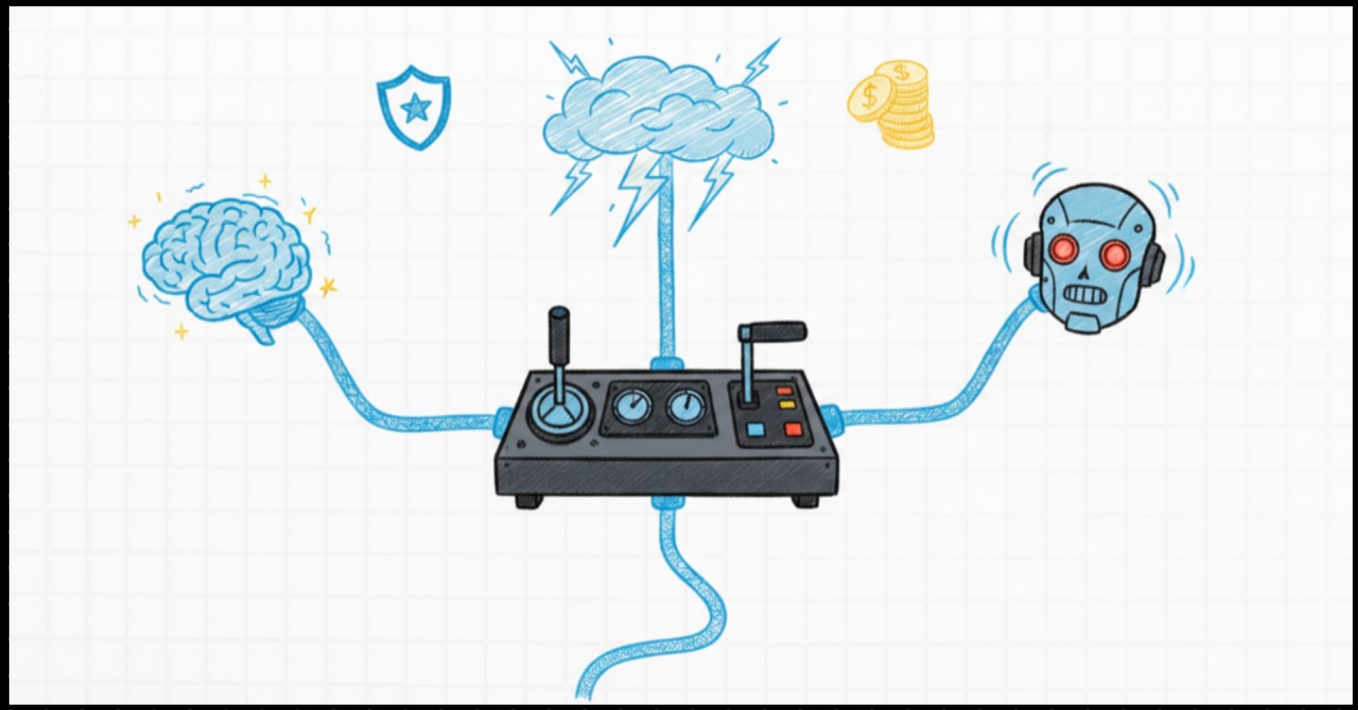


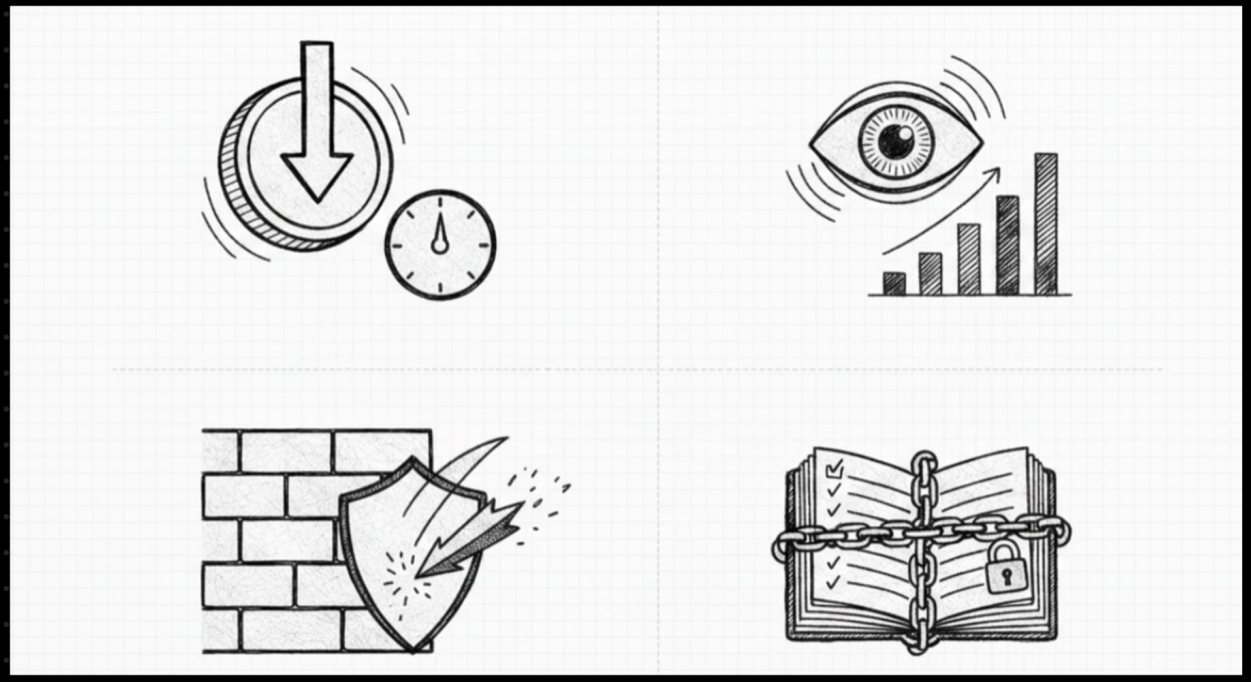

Request Flow
LLM
STOP
AI
Receive
Gateway receives the incoming request.
Route & Check
Routes request and checks circuit breakers.
Execute Call
Executes the call to the AI provider.
Fallback
Returns response or falls back if failed.


Where are your API keys right now?



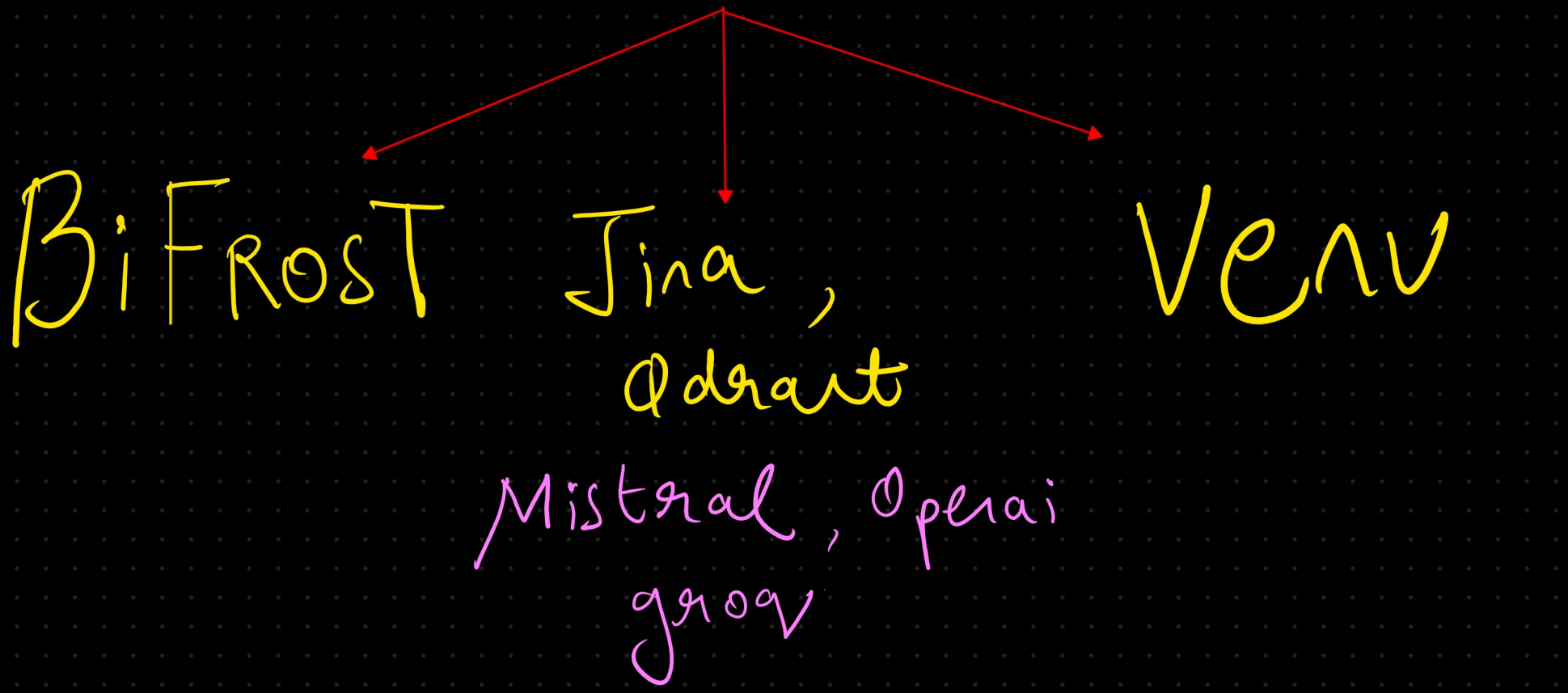
Virtual Key's






How will you manage
your AI traffic?

Setting up Bifrost and other API KEYS



Problem Without Gateway

1. Manually write fallback
2. Manage multiple Api Keys & Clients.

Whats DONE ?

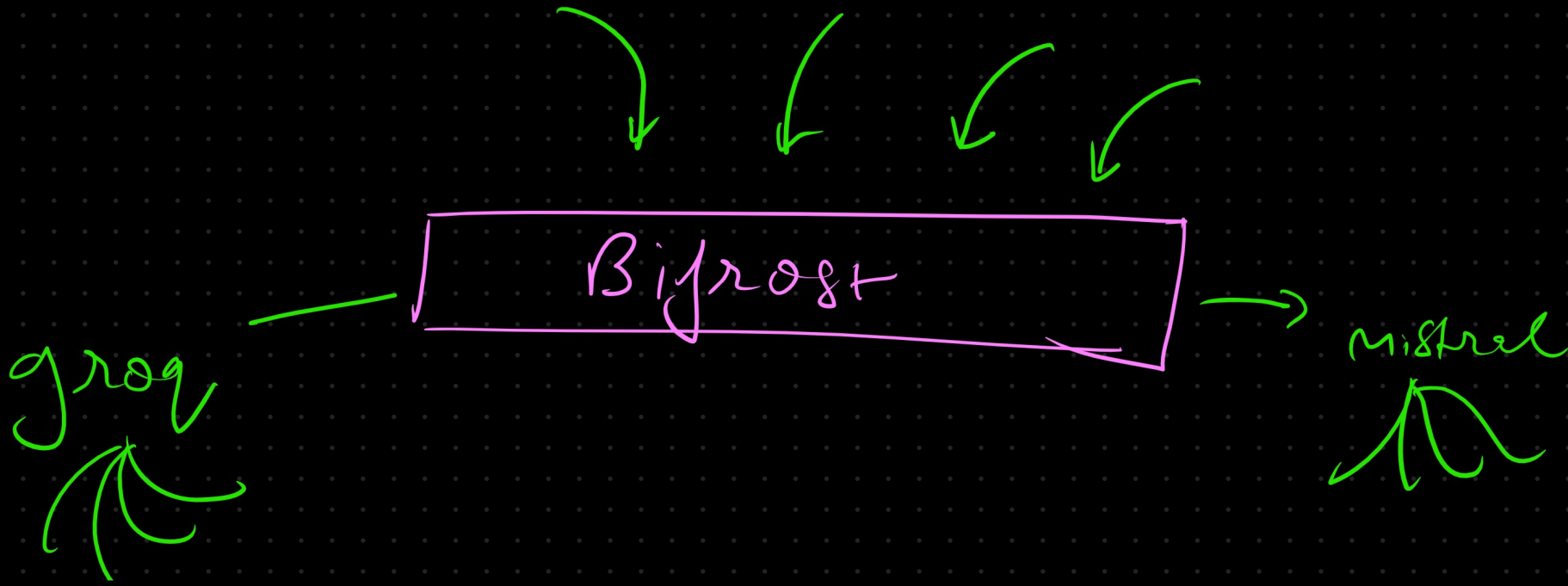
model now

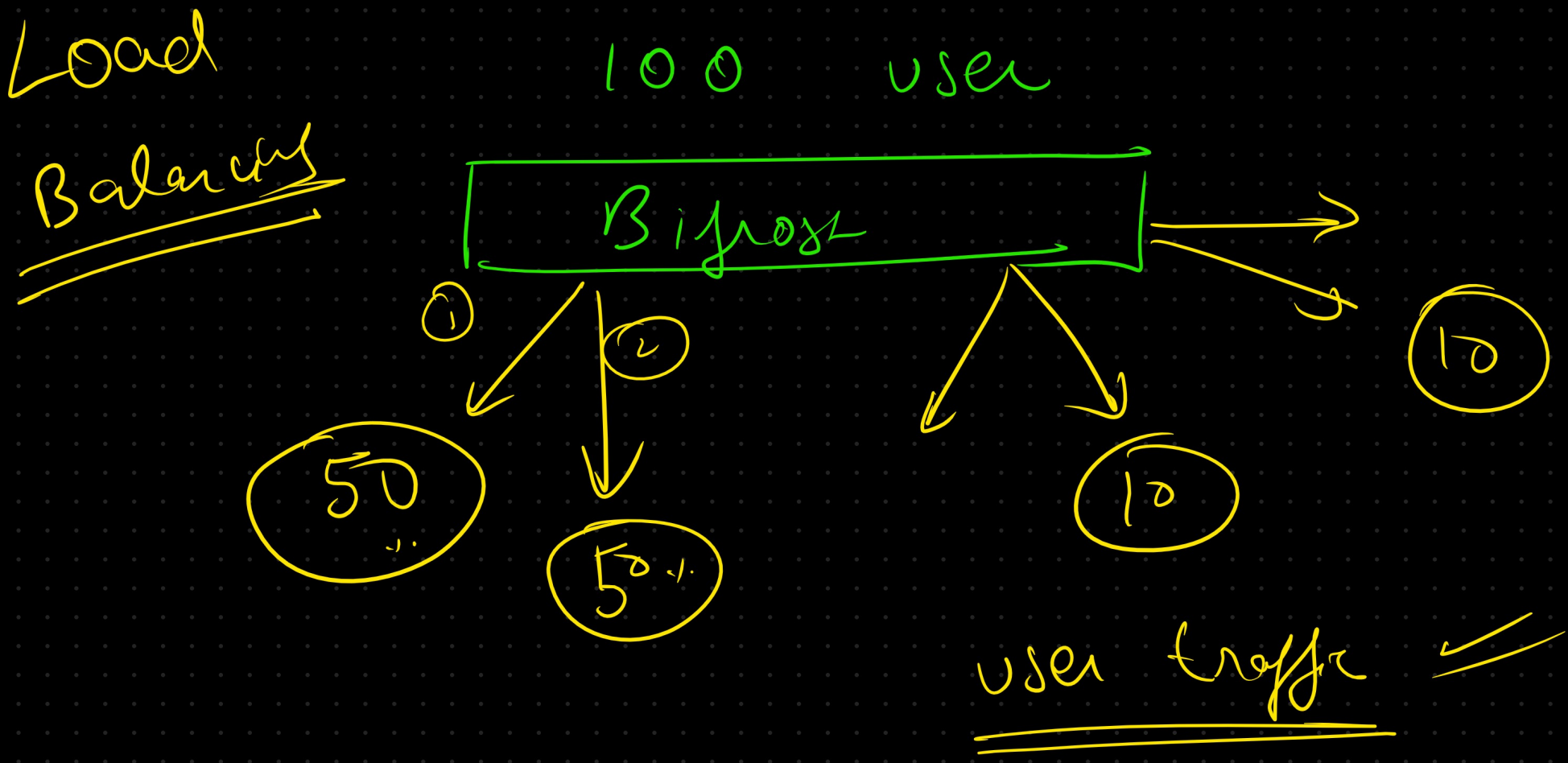
1. Bifrost

2. Calling Models
   - groq
   - mistral
   - (unified)

Agenda ahead !!

1. Automatic fallback
2. Streaming
3. Load Balancy




Load Balancer
100 user
Bifrost
1
2
10
50
50
10
user traffic

Agenda ahead !!

1. Automatic fallback ✓
2. Streaming
3. Load Balancy ✓
4. Virtual Keys ✓
5. MCP ✓
   - Deepwiki
   - Tavily

Tool

External res

gmail

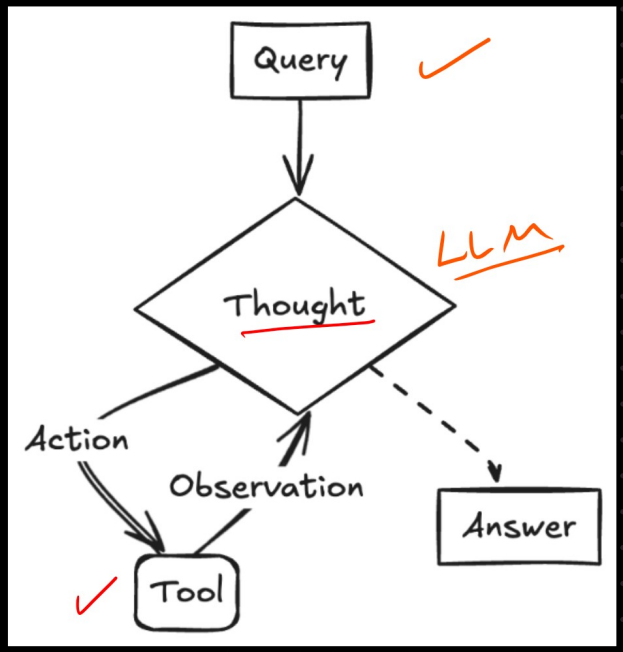
hit, Technical

Conver

DB

LLM

curosy



Tool → Tool

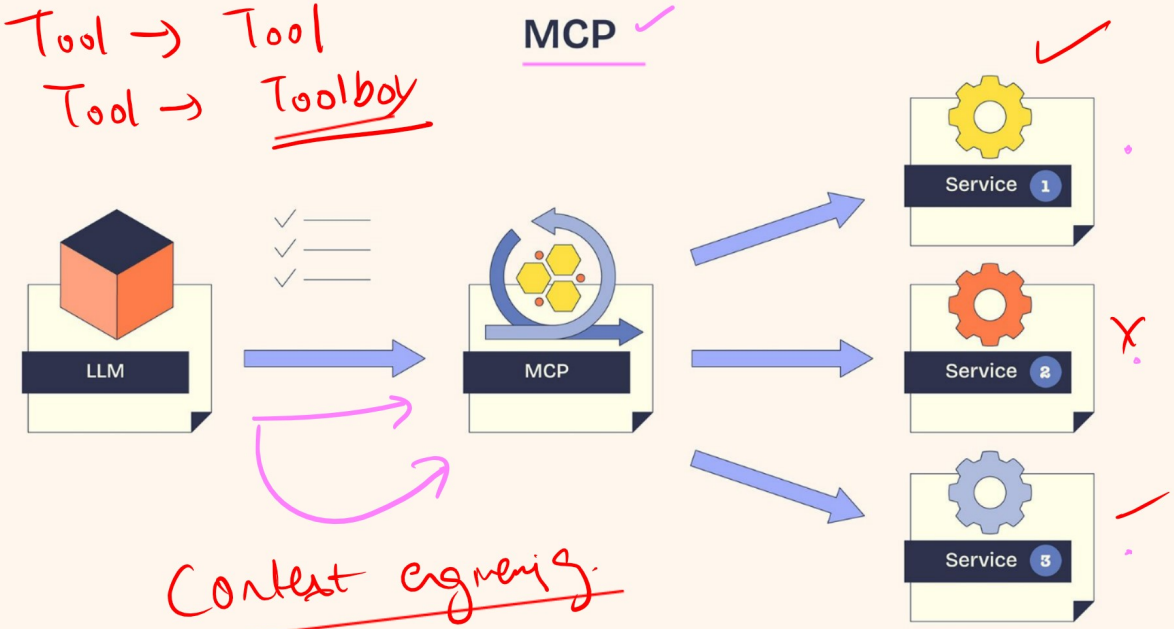
Tool → Toolbox

MCP

Tavily

web

client



Context engineering

Bifrost

Simple Rag !!

Tech Stack :-

Data ingestion → Jina ✓
Data ingestion → Qdrant ✓

Data Retrieval → Bifrost ✓

parse !!

Data

notebook !!

```
DOCS = [
    "Bifrost is a Go-based, open-source AI gateway built by Maxim
    "Bifrost supports automatic failover across providers, trigger
    "Bifrost exposes a single OpenAI-compatible /v1/chat/completic
    "Groq serves LLMs on custom LPU hardware, offering very low-la
    "Mistral AI offers La Plateforme with a free tier for models l
    "Qdrant Cloud is a managed vector database used here for RAG r
    "Jina AI provides an embeddings API (jina-embeddings-v4) with
]
```

Data our

Tavily

Field	Value
Name	`tavily`
Connection Type	`HTTP (Streamable)`
Connection URL	`https://mcp.tavily.com/mcp/?tavilyApiKey=YOUR_TAVILY_API_KEY`
Authentication Type	`None`
Code Mode Server	Off
Ping Available for Health Check	On (optional but recommended)

Replace `YOUR_TAVILY_API_KEY` with your real Tavily key.

DeepWiki

Fill in the fields as follows:

Field	Value
Name	`deepwiki`
Connection Type	`HTTP (Streamable)`
Connection URL	`https://mcp.deepwiki.com/mcp`
Authentication Type	`None`
Code Mode Server	Off
Ping Available for Health Check	On (optional but recommended)